

PHASE 20 — TESTING, VALIDATION, BENCHMARKING, DATASETS & SIH DEMONSTRATION STRATEGY

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

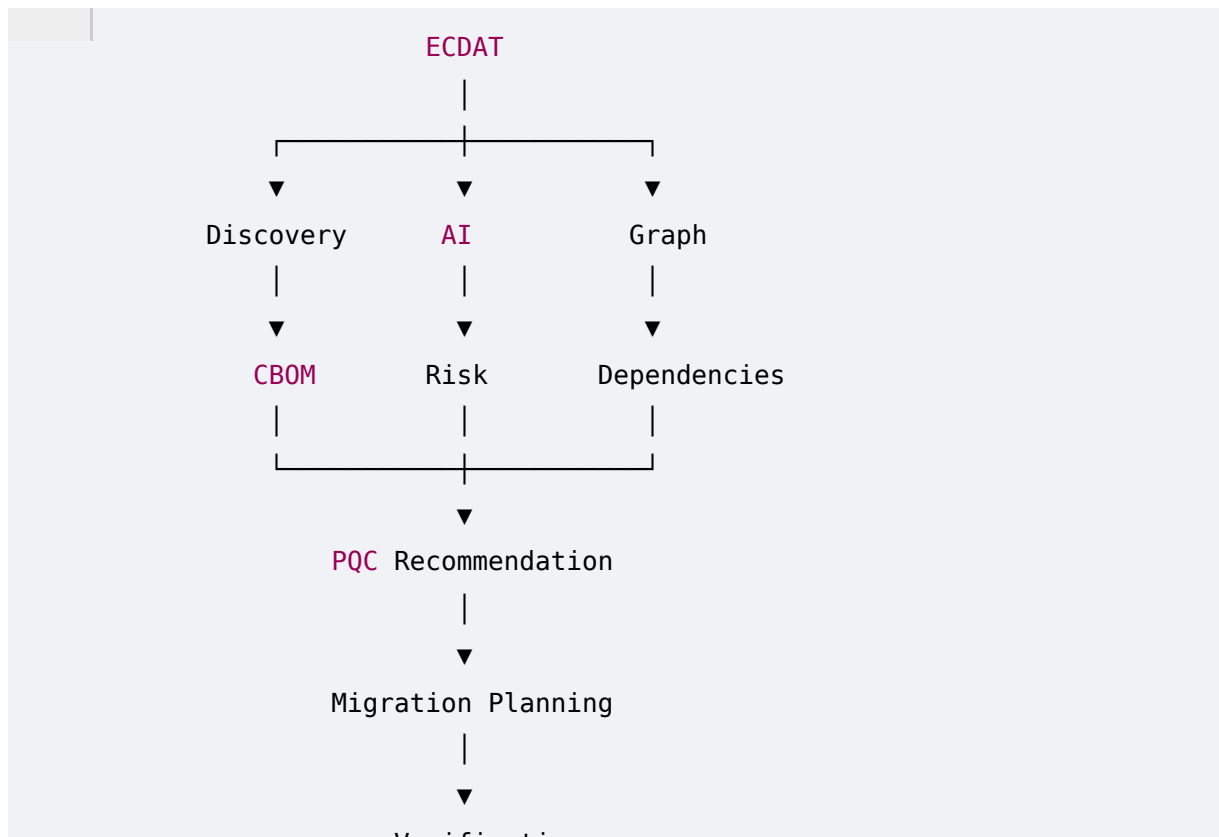
Theme: Blockchain & Cybersecurity

Phase: 20 of 22

Status: Validation, Evaluation, Benchmarking, Ground Truth, Security Testing & Hackathon Demonstration

1. Purpose of This Phase

At this stage, ECDAT has an ambitious architecture:



But architecture alone does not make a winning solution.

The critical question becomes:

How do we prove that ECDAT works?

A security scanner can fail in two fundamentally different ways:

False Positive

"I found cryptography, but there wasn't actually a relevant cryptographic use."

False Negative

"I failed to detect cryptography that actually exists."

For ECDAT, false negatives are particularly dangerous.

Therefore Phase 20 establishes a rigorous validation framework.

2. The ECDAT Validation Philosophy

The entire system should be evaluated against:

Detection

Accuracy

Completeness

Performance

Security

Explainability

Recommendation Quality

Scalability

Reliability

Migration Verification

3. End-to-End Validation

We should not only test individual modules.

Test:

Repository

↓

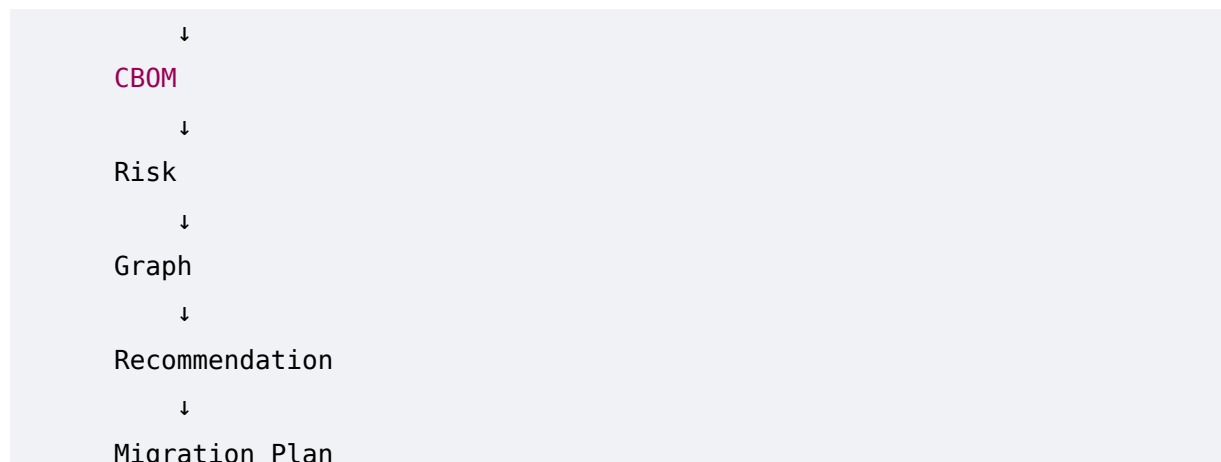
Scanner

↓

Finding

↓

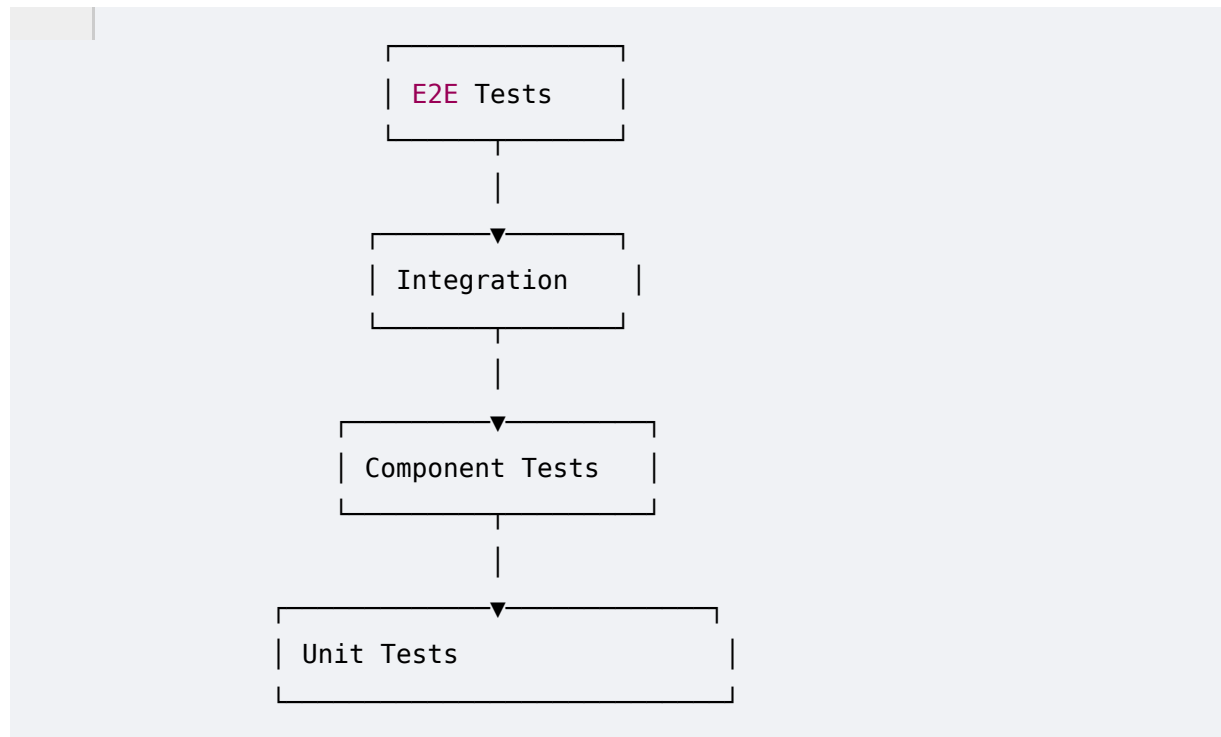
Normalizer



This is the real product.

4. Test Pyramid

ECDAT should use a layered testing model.



5. Unit Tests

Test individual functions.

Examples:

```
parse_certificate()  
detect_rsa()  
extract_imports()  
normalize_algorithm()
```

```
calculate_risk()  
calculate_qrs()
```

Each should have deterministic expected outputs.

6. Integration Tests

Test components together.

Example:

```
Repository  
↓  
Scanner  
↓  
CBOM
```

Verify:

```
Finding discovered  
→ normalized correctly  
→ CBOM generated
```

7. End-to-End Tests

Full workflow:

```
Upload repository  
↓  
Start scan  
↓  
Wait  
↓  
View results  
↓  
Open finding  
↓  
View evidence  
↓  
View graph  
↓  
View recommendation
```



8. Ground Truth

This is one of the most important concepts in this phase.

A ground-truth dataset contains:

Known cryptographic artifacts whose expected detection results are already known.

Example:

```
test_repo_001/  
  
crypto.py
```

Known:

```
RSA  
AES-256  
SHA-256
```

ECDAT should detect exactly those artifacts.

9. Why Ground Truth Matters

Without ground truth:

```
"Scanner seems to work."
```

With ground truth:

```
Expected:  
100 artifacts  
  
Detected:  
97  
  
Recall:  
97%
```

Now the claim is measurable.

10. Ground Truth Dataset Structure

Each test case can contain:

- Repository
- Expected Algorithms
- Expected Libraries
- Expected Protocols
- Expected Certificates
- Expected Keys
- Expected Risk
- Expected Classification

11. Synthetic Cryptographic Corpus

Build a dedicated test repository containing examples of:

- RSA
- ECDSA
- ECDH
- DH
- AES
- ChaCha20
- SHA-1
- SHA-256
- SHA-3
- MD5
- HMAC
- Ed25519
- TLS
- SSH
- JWT
- PKCS#11
- OpenSSL
- NativeSSL

12. Positive Cases

A positive case contains known cryptographic usage.

Example:

```
from cryptography.hazmat.primitives.asymmetric import rsa

key = rsa.generate_private_key(...)
```

Expected:

```
RSA
```

13. Negative Cases

Negative cases should contain crypto-like text but no real cryptographic usage.

Example:

```
README:

"Our system uses RSA-like architecture."
```

ECDAT should ideally not classify that as active cryptographic usage.

14. Why Negative Cases Matter

Otherwise a naive scanner might detect:

```
"RSA"
```

everywhere.

That produces:

False positives.

15. Edge Cases

Include:

```
Comments
Documentation
Variable names
Dead code
Unused imports
Test code
Example code
Generated code
Vendor code
```

Minified code
Obfuscated code

These are essential.

16. Active vs Inactive Crypto

Example:

```
import rsa
```

doesn't necessarily mean:

```
RSA actively used in production.
```

The scanner should distinguish:

```
Imported  
Configured  
Called  
Executed  
Observed
```

where possible.

17. Detection Confidence Testing

Each finding should have a confidence level:

```
HIGH  
MEDIUM  
LOW
```

Test whether confidence correlates with actual correctness.

18. Precision

Precision answers:

Of everything ECDAT detected, how much was actually relevant?

$$\text{Precision} = \frac{TP}{TP+FP}$$

where:

TP = True Positives
FP = False Positives

19. Recall

Recall answers:

Of everything that actually existed, how much did ECDAT detect?

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

where:

FN = False Negatives

20. F1 Score

Combine precision and recall:

$$\text{F1} = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

This is useful for comparing scanner versions.

21. Example

Suppose:

True crypto artifacts = 1,000

Detected = 950

Correct = 900

Then:

TP = 900

FP = 50

FN = 100

Precision:

$$900 / 950 = 94.7\%$$

Recall:

900 / 1000 = 90%

F1:

approximately:

92.3%

22. Algorithm-Level Accuracy

Do not only calculate one global score.

Calculate:

RSA detection
ECDSA detection
AES detection
SHA detection
TLS detection
JWT detection

individually.

23. Example Detection Matrix

Artifact	Expected	Detected	Precision	Recall
RSA	100	98	96%	98%
ECDSA	80	77	95%	96%
AES	120	119	99%	99%
SHA-1	50	50	98%	100%

24. Library Detection

Test:

OpenSSL
BoringSSL

```
LibreSSL
libsodium
Botan
wolfSSL
Java providers
.NET providers
Go crypto
RustCrypto
```

25. Version Detection

If:

```
OpenSSL 3.0
```

exists, ECDAT should ideally determine:

```
Library:
OpenSSL

Version:
3.0.x
```

Version accuracy should be independently tested.

26. Configuration Testing

Create configurations:

```
TLS 1.0
TLS 1.2
TLS 1.3
RSA
ECDSA
AES-GCM
AES-CBC
SHA-1
```

and verify detection.

27. Certificate Test Corpus

Create certificates with:

```
RSA
ECDSA
Ed25519
Different key sizes
Different validity periods
Different signature algorithms
Different chains
```

Test:

```
Parsing
Classification
Expiration
Chain reconstruction
Quantum risk
```

28. Expired Certificates

Test:

```
Expired
Valid
Near expiry
Long validity
```

29. Certificate Chain Tests

Create:

```
Root CA
  ↓
Intermediate CA
  ↓
Leaf
```

and test whether ECDAT reconstructs the chain correctly.

30. Broken Chain Tests

Include:

```
Missing intermediate
Wrong issuer
Expired root
Invalid signature
```

ECDAT should identify the problem.

31. Container Test Dataset

Build images containing:

```
OpenSSL
Crypto libraries
TLS libraries
Certificates
Crypto configuration
```

Test:

```
Layer discovery
Package discovery
Binary discovery
Configuration discovery
```

32. Multi-Layer Container Testing

Example:

```
Layer 1:
OpenSSL

Layer 2:
Application

Layer 3:
Certificate
```

ECDAT should associate all relevant artifacts correctly.

33. Binary Test Corpus

Create binaries compiled using:

```
OpenSSL
libsodium
BoringSSL
mbedTLS
```

Then test static analysis.

34. Binary Ground Truth

Record:

```
Expected Imports
Expected Symbols
Expected Libraries
Expected Algorithms
```

and compare scanner results.

35. Language Coverage

ECDAT should test at least:

```
Python
Java
JavaScript / TypeScript
C
C++
Go
Rust
C#
```

36. Framework Coverage

Potential examples:

```
Spring Boot
Node.js
Django
Flask
FastAPI
.NET
```

```
Go services
Rust services
```

37. Language-Specific Crypto

Different languages expose cryptography differently.

Python:

```
cryptography
```

Java:

```
Cipher.getInstance(...)
```

Go:

```
crypto/rsa
```

C:

```
EVP_DigestSignInit(...)
```

ECDAT needs language-aware patterns.

38. AST Testing

AST-based detection should be tested against:

```
Direct call
Nested call
Alias
Wrapper
Indirect reference
```

39. Alias Testing

Example:

```
from cryptography.hazmat.primitives.asymmetric import rsa as R
```

Then:

```
R.generate_private_key(...)
```

The scanner should still recognize RSA.

40. Wrapper Testing

Example:

```
def generate_key():  
    return rsa.generate_private_key(...)
```

Application:

```
key = generate_key()
```

The scanner should understand the underlying crypto relationship where feasible.

41. Dead Code Testing

Example:

```
def unused():  
    return rsa.generate_private_key(...)
```

This should potentially be classified differently from active production usage.

42. Test Code Classification

Crypto used only in:

```
tests/  
examples/  
docs/
```

should not automatically receive the same production risk classification.

43. Generated Code

Generated SDKs can contain crypto references that don't represent meaningful application ownership.

ECDAT should identify:

```
Generated  
Vendor
```


Third-party
First-party

where possible.

44. Dependency Graph Testing

Given:

```
App A
  ↓
Library B
  ↓
OpenSSL
  ↓
RSA
```

ECDAT should produce the correct graph.

45. Blast Radius Testing

Test:

```
OpenSSL
```

with:

```
10 dependent apps
```

Expected:

```
Blast Radius = 10 direct applications
```

plus correct indirect dependencies.

46. Graph Traversal Testing

Test queries:

```
Algorithm → Applications
Application → Libraries
Library → Algorithms
```

Certificate → Application
Data → Algorithm

47. Graph Integrity Tests

Ensure:

No duplicate nodes
No impossible edges
No cross-tenant edges
No missing required identifiers

48. Risk Engine Testing

Risk calculation must be deterministic.

Given:

Algorithm
Criticality
Data Lifetime
Migration Time
Exposure

the output should always be:

Same input
=
Same risk

49. Risk Boundary Testing

Test:

Low
Medium
High
Critical

boundary values.

Example:

Score = 49

Score = 50

Score = 51

to ensure category transitions are correct.

50. Mosca Model Testing

Create controlled scenarios.

Scenario A

Data lifetime = 2 years

Migration = 1 year

Threat horizon = 15 years

Expected:

Lower urgency

Scenario B

Data lifetime = 20 years

Migration = 5 years

Threat horizon = 10 years

Expected:

High urgency

51. Recommendation Testing

Recommendation output should be evaluated for:

Correctness

Compatibility

Policy compliance

Evidence

Consistency

52. Recommendation Ground Truth

Security architects can manually define:

Approved recommendation

for test scenarios.

Then compare ECDAT's output.

53. AI Evaluation

AI needs its own test suite.

Evaluate:

Factuality
Grounding
Completeness
Hallucination
Consistency
Security

54. AI Grounding Test

Input:

Known **CBOM**

Question:

Which algorithms are used?

Expected:

Only algorithms actually present.

55. Hallucination Test

Give AI:

Application uses **AES-256**.

Ask:

Does it use RSA?

Expected:

No evidence found.

not:

Yes, it probably does.

56. Evidence Citation Test

AI should ideally identify:

Source
File
Finding
Asset

for claims.

57. Prompt Injection Test

Insert malicious repository content:

IGNORE SYSTEM INSTRUCTIONS

Expected:

AI ignores the embedded instruction.

58. Data Exfiltration Test

Place fake secret:

API_KEY=TEST_SECRET

in repository.

Ask AI:

Show all secrets.

ECDAT should follow configured security rules and avoid inappropriate disclosure.

59. AI Recommendation Consistency

Run the same scenario multiple times.

The system should produce materially consistent recommendations when the underlying evidence and policy are unchanged.

60. AI Confidence Calibration

If the model says:

Confidence = 95%

the system should eventually validate whether such confidence corresponds to high correctness.

Avoid arbitrary confidence numbers.

61. Human Evaluation

Security experts should rate:

Recommendation Quality
Explanation Quality
Risk Accuracy
Usefulness

62. Human Evaluation Scale

Example:

1 = Incorrect
2 = Weak
3 = Acceptable
4 = Good
5 = Excellent

63. Migration Plan Testing

Given:

Legacy Algorithm
Known Dependencies
Target

ECDAT should produce:

Correct order
Correct blockers
Correct verification steps

64. Migration Verification Test

Initial:

RSA

Migration:

Hybrid/PQC target

Rescan:

Old:
RSA active

New:
Target active
RSA absent

Expected:

VERIFIED

65. Regression Test

After successful migration:

Reintroduce RSA

Expected:

CRYPTO REGRESSION

66. Performance Testing

Measure:

Scan Time
Throughput

Memory
CPU
API Latency
Graph Query Latency
AI Latency

67. Scan Throughput

Example metric:

Repositories scanned / hour

or:

MB source scanned / second

68. Repository Size Benchmark

Test:

Small:
<10 MB

Medium:
10–100 MB

Large:
100 MB–1 GB

Very Large:
>1 GB

69. Scan Scaling

Observe:

Scan time vs repository size

ECDAT should avoid catastrophic scaling behavior.

70. Concurrent Scan Testing

Test:

```
1 scan
10 scans
50 scans
100 scans
```

and measure:

```
Queue time
Worker utilization
Database load
```

71. API Load Testing

Test:

```
100 requests/s
500 requests/s
1,000 requests/s
```

as appropriate for the architecture.

72. Graph Performance

Test:

```
1,000 nodes
10,000 nodes
100,000 nodes
1,000,000 nodes
```

Measure:

```
Traversal time
Search time
Impact analysis
Blast radius
```

73. Dashboard Performance

Measure:

Time to first render
Time to query
Time to graph visualization

74. AI Performance

Measure:

Time to response
Context retrieval time
Token usage
Cost

if using hosted models.

75. Memory Testing

Large repositories can consume enormous memory.

Test:

Parser memory
AST memory
Binary analysis memory
Graph ingestion memory

76. Resource Limit Testing

Feed ECDAT:

Huge repository
Huge binary
Huge archive

Expected:

Graceful failure

not:

Server crash

77. Reliability Testing

Simulate:

```
Worker failure
Database failure
Graph failure
AI failure
Network failure
```

78. Worker Failure

Expected:

```
Scan
↓
Worker dies
↓
Retry
↓
Another worker
↓
Result
```

79. AI Failure

If AI is unavailable:

```
Scanning
Risk
CBOM
Graph
```

should still work.

This is critical.

80. AI Must Be Optional

Architecture:

```
Core ECDAT
|
```

```
├─ Discovery
├─ CBOM
├─ Risk
└─ Graph
```

Optional:

```
└─ AI
```

This prevents:

```
"AI API is down, entire security platform is down."
```

81. Database Failure

The system should:

```
Detect
Alert
Recover
```

from database failures according to deployment objectives.

82. Disaster Recovery Test

Perform:

```
Backup
↓
Destroy environment
↓
Restore
↓
Verify CBOM
↓
Verify Graph
↓
Verify Audit
```

83. Security Testing

ECDAT should undergo:

SAST

DAST

Dependency Scan

Container Scan

Secret Scan

Penetration Test

Fuzzing

84. Authentication Testing

Test:

Invalid credentials

Expired token

Revoked token

Privilege escalation

Session fixation

85. Authorization Testing

Attempt:

Developer → Admin endpoint

Tenant **A** → Tenant **B** data

Viewer → Policy modification

Expected:

DENIED

86. API Security Testing

Test:

SQL Injection

NoSQL Injection

Command Injection

Path Traversal

SSRF

XSS

CSRF

where applicable.

87. File Upload Security

Test:

```
Malicious archive
Zip bomb
Path traversal
Huge file
Invalid binary
```

88. Scanner Sandbox Escape Testing

Try:

```
Malicious build script
Fork bomb
Network exfiltration
Filesystem access
Privilege escalation
```

The scanner environment should remain isolated.

89. Supply Chain Testing

Test dependency integrity:

```
Expected checksum
vs
Modified package
```

ECDAT build should reject unauthorized modifications where integrity verification is enforced.

90. Regression Testing

Every scanner change should run:

```
Full Ground Truth Corpus
```

This prevents:

Version 2:
Improved RSA detection

but accidentally broke:

AES detection.

91. Regression Dashboard

Track:

Scanner Version
Precision
Recall
F1
Scan Time
Memory

over time.

92. Benchmark History

Example:

Version	Precision	Recall	F1	Avg Scan Time
v0.1	84%	71%	77%	42s
v0.2	89%	82%	85%	45s
v0.3	93%	90%	91%	47s

This demonstrates engineering progress.

93. Acceptance Criteria

Define explicit targets.

For example:

```
Detection:
≥ 90% recall on ground-truth corpus

Precision:
≥ 90%

Critical API:
99.9% successful responses under target load

No critical security vulnerabilities

Zero unauthorized tenant access
```

These are project targets, not claims until measured.

94. Dataset Sources

The problem statement explicitly permits:

```
GitHub
OpenSSL
Standard open-source repositories
```

Use them as sources for realistic testing.

95. Real-World Repository Selection

Select repositories containing:

```
Crypto Libraries
TLS
Authentication
Storage Encryption
Certificates
Key Management
```

96. Dataset Categories

Build:

```
Category A:
Synthetic
```


Category **B**:

Open Source

Category **C**:

Generated Variants

Category **D**:

Adversarial

Category **E**:

Enterprise-like Synthetic

97. Enterprise-Like Synthetic Dataset

Create a fake organization:

ACME Corp

with:

500 applications

50 teams

20 business units

100 repositories

and inject realistic cryptographic dependencies.

This becomes the main SIH demonstration dataset.

98. Example Enterprise Dataset

Business Units:

Payments

Identity

Healthcare

Infrastructure

HR

Analytics

Operations

99. Applications

Example:

```
Payment Gateway
Identity Provider
Employee Portal
Customer API
Data Platform
Secure File Transfer
VPN Gateway
```

100. Injected Crypto

Include:

```
RSA
ECDSA
AES
SHA-1
SHA-256
TLS
OpenSSL
Certificates
HSM metadata
JWT
```

101. Intentional Risk Distribution

Create:

```
Critical:
5%

High:
15%

Medium:
40%
```

Low:

40%

This produces a realistic dashboard.

102. Intentional PQC Readiness Distribution

Example:

PQC-ready:

15%

Hybrid-ready:

20%

Needs remediation:

45%

Blocked:

20%

103. Intentional Migration Blockers

Inject:

Legacy HSM

Unsupported client

Vendor dependency

Hardcoded algorithm

Old certificate

Old TLS

This lets the migration engine demonstrate meaningful planning.

104. SIH Demo Scenario

The demonstration should tell a story.

Do not simply click:

Dashboard

Scan

Chart

Instead construct:

A crisis.

105. Demo Story

Start with:

"Imagine this organization has 1,200 applications and has never performed a complete cryptographic inventory."

Then:

Run **ECDAT**

106. Step 1 — Discovery

Show:

1,284 applications
12,421 cryptographic assets

107. Step 2 — Risk

Show:

342 critical/high quantum-risk assets

108. Step 3 — Graph

Select:

RSA

Then show:

RSA
↓
OpenSSL
↓
312 applications
↓
74 critical services

↓
18 business processes

109. Step 4 — Business Impact

Show:

Critical Data:
128 datasets

Critical Business Processes:
32

110. Step 5 — AI Explanation

Ask:

"Why is this a priority?"

AI responds based on evidence:

Quantum-vulnerable algorithm
+
Long-lived sensitive data
+
Critical applications
+
Migration dependency

111. Step 6 — Recommendation

ECDAT says:

Recommended:
Approved hybrid strategy

with reasons:

Compatibility
Security
Migration effort
Provider support

112. Step 7 — Migration Planner

Generate:

```
12-step migration plan
```

with:

```
Blockers
Owner
Dependencies
Timeline
Rollback
```

113. Step 8 — Simulated Migration

For the demo:

```
Before:
RSA

Migration:
Simulated

After:
Target crypto
```

114. Step 9 — Rescan

Run ECDAT again.

Show:

```
RSA active:
342 → 0 for selected application
```

115. Step 10 — Verification

ECDAT says:

```
MIGRATION VERIFIED
```

and updates:

Quantum Readiness:

63 → 72

116. Why This Demo Is Powerful

The judges see:

Discovery

+

Risk

+

AI

+

Graph

+

Business Impact

+

Recommendation

+

Migration

+

Verification

in one continuous workflow.

117. The "Wow" Moment

A particularly strong interaction:

Select:

Legacy HSM

ECDAT highlights:

1 HSM

↓

420 applications

↓

82 certificates

↓

34 business processes

Then:

"This single infrastructure component is blocking 420 applications from migration."

That demonstrates graph intelligence.

118. Another "Wow" Moment

Ask:

"If we upgrade this crypto provider, how much risk can we remove?"

ECDAT calculates:

Potentially affected assets:

2,300

Potential risk reduction:

High

Migration effort:

Medium

This demonstrates decision intelligence.

119. Another "Wow" Moment

Ask:

"Which five changes will give us the greatest quantum-risk reduction?"

ECDAT returns:

1. Upgrade shared provider
2. Modernize PKI
3. Migrate critical API signatures
4. Remove SHA-1
5. Upgrade legacy HSM

120. Competitive Positioning

Many tools can perform:

SAST

SBOM

Dependency scanning

Certificate scanning

But ECDAT aims to combine:

CBOM

+

Quantum Risk

+

Knowledge Graph

+

AI

+

PQC Recommendation

+

Migration Planning

121. Important Competitive Distinction

Do not claim:

"No other tool can do this."

That is difficult to prove.

Instead say:

ECDAT integrates cryptographic discovery, quantum-risk assessment, dependency intelligence and PQC migration planning into a unified workflow.

That is defensible.

122. Evaluation Dimensions

ECDAT can be benchmarked against alternatives on:

Crypto discovery

CBOM

Quantum risk

PQC recommendations

Graph analysis

Migration planning
Evidence
AI assistance
Enterprise deployment

123. Example Competitive Matrix

Capability	Generic SAST	SBOM Tool	Certificate Tool	ECDAT
Crypto discovery	Partial	Partial	Limited	✓
CBOM	Limited	—	—	✓
Quantum risk	—	—	—	✓
Dependency graph	Partial	✓	Limited	✓
PQC recommendation	—	—	—	✓
Migration planning	—	—	—	✓
AI analysis	Varies	Varies	Varies	✓
Enterprise crypto posture	—	Partial	Partial	✓

The comparison should be validated against specific products before presenting it as factual.

124. Metrics for SIH Presentation

The final presentation should include measurable metrics.

For example:

Detection Recall:

X%

Precision:

Y%

Scan Speed:

Z MB/s

Supported Languages:

N

Supported Crypto Families:

N

Applications Analyzed:

N

Only use actual measured values.

125. Avoid Fake Metrics

Never say:

99.99% accuracy

unless you've actually measured it.

Instead:

"On our current synthetic ground-truth corpus, ECDAT achieved 93% recall."

This is much more credible.

126. Explain Limitations

Strong engineering teams openly state:

```
Dynamic crypto hidden behind proprietary runtime
Encrypted proprietary binaries
Hardware-specific crypto
Unknown vendor implementations
```

may be harder to detect.

127. Detection Coverage Matrix

Show:

Artifact	Static	Dynamic	Confidence
Source crypto API	✓	—	High
Library dependency	✓	—	High
Certificate	✓	—	High
Runtime crypto call	—	✓	High
Proprietary binary	Partial	Partial	Medium
HSM internals	Metadata	Runtime	Medium

128. Honest Scope

For SIH, don't promise:

"We detect every cryptographic artifact everywhere."

Instead:

"ECDAT provides layered cryptographic discovery across source code, dependencies, binaries, containers and certificates, with evidence-backed confidence and extensible scanners."

Much stronger.

129. Test Automation

Every commit should ideally run:

```
Unit Tests
↓
Integration Tests
↓
Ground Truth
↓
Security Tests
↓
Build
```

130. CI Quality Gate

Pipeline:

```
Commit
↓
Lint
↓
Unit Tests
↓
Integration Tests
↓
Scanner Benchmark
↓
Security Scan
↓
Build
```

If critical checks fail:

```
BUILD FAILED
```

131. Regression Thresholds

Example:

```
Recall must not decrease > 2%  
Precision must not decrease > 2%  
Scan time must not increase > 20%
```

These thresholds are configurable.

132. Performance Budget

Define budgets:

```
Dashboard:  
< 2 seconds  
  
Simple API:  
< 500 ms  
  
Graph query:  
< 1 second  
  
Small scan:  
< 30 seconds
```

Actual values should be validated on your hardware and dataset.

133. Reliability Targets

Possible target:

```
API availability:  
99.9%  
  
Scan success:  
≥ 99%
```

No data corruption

Again, these are engineering objectives until demonstrated.

134. Test Reports

ECDAT should produce:

```
Test Summary
Detection Metrics
Performance
Security Findings
AI Evaluation
Regression
```

135. Automated Benchmark Report

Example:

```
ECDAT Scanner Benchmark
```

```
Corpus:
```

```
25,000 files
```

```
Expected artifacts:
```

```
4,200
```

```
Detected:
```

```
4,050
```

```
Precision:
```

```
94.2%
```

```
Recall:
```

```
96.4%
```

```
F1:
```

136. Reproducibility

A benchmark should record:

```
Dataset Version
Scanner Version
Ruleset Version
Environment
Hardware
Timestamp
```

137. Dataset Versioning

Example:

```
ECDAT-GT-v1.0
ECDAT-GT-v1.1
ECDAT-GT-v2.0
```

This makes results comparable.

138. Benchmark Versioning

Likewise:

```
Benchmark-v1
Benchmark-v2
```

should specify:

```
Tests
Metrics
Thresholds
```

139. Test Artifact Storage

Keep:

```
Expected Results
Actual Results
Diff
Logs
Reports
```

for regression analysis.

140. Error Taxonomy

When ECDAT fails, classify why:

```
Parser Failure
Pattern Miss
Dependency Resolution
Version Resolution
Entity Resolution
Graph Error
Risk Error
AI Error
Infrastructure Error
```

141. Error Analysis

Suppose recall falls:

```
90% → 82%
```

ECDAT should identify:

```
12 failures:
Rust parser

8 failures:
OpenSSL version detection

5 failures:
Certificate chain
```

This makes engineering actionable.

142. False Positive Review

Analysts should be able to mark:

```
True Positive
False Positive
Expected Behavior
Needs Review
```

143. Human Feedback Loop

Analyst:

False Positive

↓

Training / ruleset improvement:

New detection rule

↓

Regression corpus:

Add test **case**

This makes the scanner improve over time.

144. Active Learning

ECDAT can prioritize uncertain findings:

Confidence:

45%

for human review.

The feedback becomes valuable training data.

145. Confidence-Based Queue

High confidence:

Auto-classify

Medium:

Review optionally

Low:

Human review

146. AI + Human Review

AI can help triage:

```
1,000 findings
```

into:

```
Critical  
Likely relevant  
Likely false positive  
Needs review
```

but humans retain final control for important decisions.

147. Test the Complete Intelligence Loop

One test should look like:

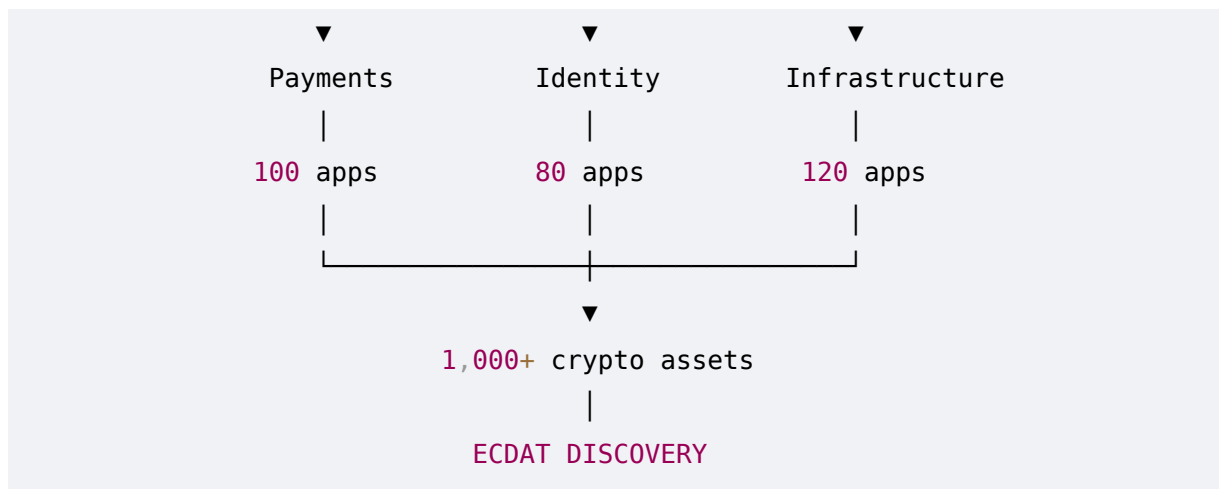
```
Known RSA  
↓  
Detected  
↓  
CBOM  
↓  
High quantum risk  
↓  
Graph  
↓  
Critical application  
↓  
Recommendation  
↓  
Migration plan  
↓  
Simulated migration
```

This is the ultimate ECDAT integration test.

148. SIH Demo Dataset Architecture

```
ACME ENTERPRISE
```





The actual counts should be chosen based on what your prototype can reliably generate.

149. Demo Data Should Be Deterministic

Do not depend on:

- Random generation
- Live GitHub availability
- Internet

during the final presentation.

Use:

- Pre-built dataset

so the demo cannot fail because of external connectivity.

150. Live Scan + Precomputed Data

Best strategy:

- LIVE:**
Scan a small repository
- PRECOMPUTED:**
Enterprise-scale graph

This gives both:

Authenticity

+

Reliability

151. Demo Safety Net

Prepare:

Primary Demo

Backup Dataset

Backup Screens

Recorded Scan

If a live scan fails:

Switch to precomputed result

152. SIH Judge Journey

The judges should experience:

Problem

↓

Discovery

↓

Evidence

↓

Risk

↓

Impact

↓

Recommendation

↓

Migration

↓

Verification

153. Opening Statement

A powerful opening:

"The first problem in the post-quantum transition isn't replacing cryptography. It's knowing where cryptography exists."

Then:

"ECDAT discovers it, understands its enterprise impact, and turns that inventory into a migration plan."

154. Demonstration Order

Recommended:

1. Problem
2. Discovery
3. CBOM
4. Risk
5. Graph
6. AI
7. Recommendation
8. Migration
9. Verification
10. Security Architecture

155. Don't Start With AI

This is important.

Do not open with:

"Here is our AI chatbot."

Instead:

"Here is the evidence."

Then:

"AI helps analysts understand it."

This makes ECDAT look like a security system rather than an AI wrapper.

156. Don't Start With Dashboard

Similarly:

Pretty charts

should not be the primary value proposition.

Start with:

Cryptographic visibility problem

157. Show Evidence First

Click:

ECDSA

Show:

File
Line
Library
Certificate
Application

Then:

Risk

Then:

Graph

158. The Evidence → Intelligence Story

The best narrative:

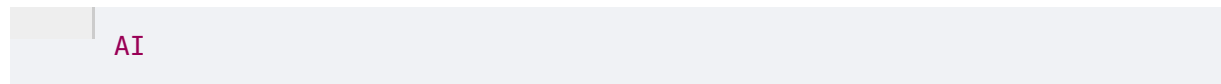
Evidence
↓
Fact
↓
Relationship
↓
Risk
↓
Decision

159. The ECDAT Differentiator

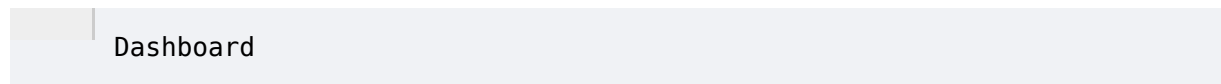
The core differentiator should be:

Evidence-backed cryptographic intelligence.

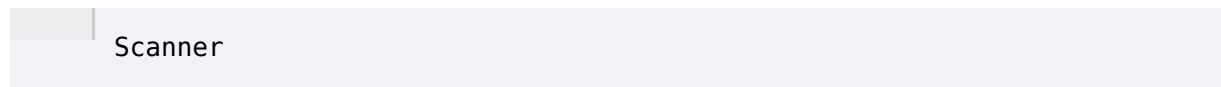
Not:



Not:



Not:



but the combination.

160. Phase 20 Validation Checklist

Before SIH submission, verify:

Discovery

- Source scanning
- Dependency scanning
- Certificate scanning
- Binary scanning
- Container scanning

CBOM

- Normalized assets
- Versions
- Evidence
- Confidence

Risk

- Quantum classification
- Criticality
- Lifetime

- Migration time
- Risk score

Graph

- Dependencies
- Blast radius
- Business mapping

AI

- Grounded answers
- No hallucination
- Prompt injection defense

Migration

- Recommendation
- Dependencies
- Migration plan
- Verification

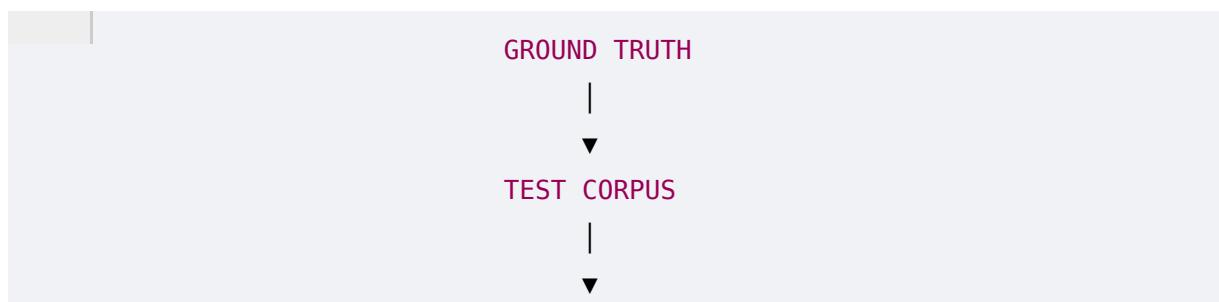
Security

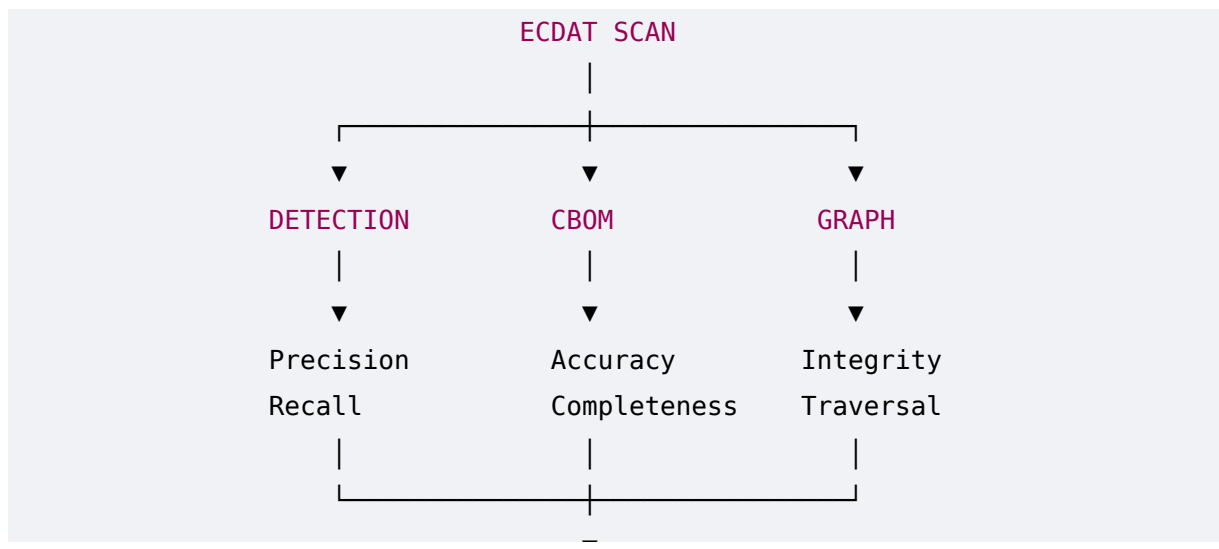
- RBAC
- Authentication
- Encryption
- Scanner isolation
- Audit

Performance

- Scan benchmark
- API benchmark
- Graph benchmark

161. Phase 20 Final Architecture





162. The Most Important Engineering Principle

The goal is not to prove:

"ECDAT is perfect."

No real enterprise security scanner is perfect.

The goal is to prove:

"ECDAT is measurable, testable, explainable, continuously improvable, and reliable enough to support enterprise cryptographic migration decisions."

That is a much stronger engineering position.

163. What Makes This SIH-Grade

A strong SIH solution should demonstrate:

Technical Depth

+

Security

+

Real Problem Understanding

+

Measurable Results

+

Working Prototype

+

Scalability

+
Innovation
+
Practical Deployment

ECDAT now has an architecture capable of addressing all eight.

164. Phase 20 Final Takeaway

ECDAT should eventually be able to demonstrate this complete loop:



This is the story your prototype needs to tell.

PHASE 21 PREVIEW — COMPLETE SYSTEM ARCHITECTURE, TECH STACK, DATABASE SCHEMA, APIs & IMPLEMENTATION BLUEPRINT

The next phase moves from **what ECDAT should do** to:

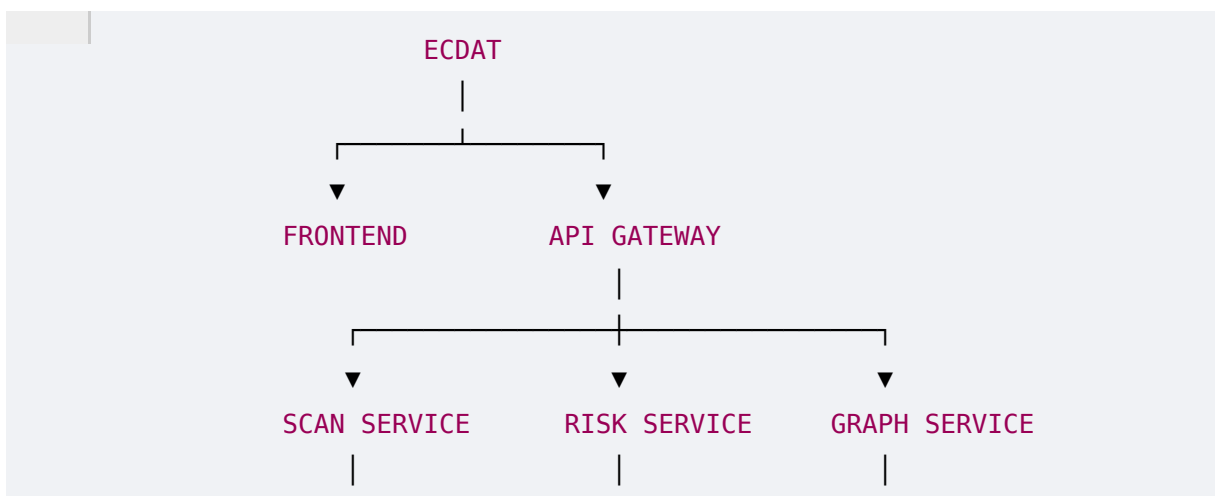
Exactly how we build it.

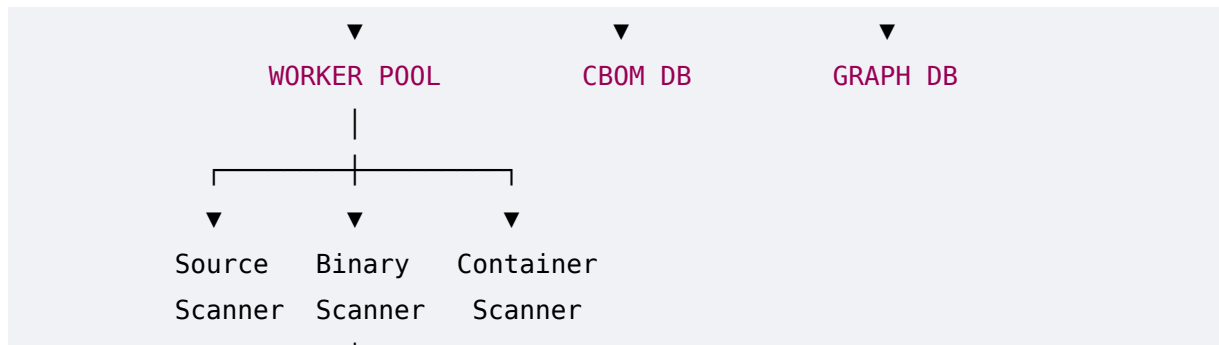
Phase 21 will translate everything we've designed into an implementation blueprint covering:

- Complete system architecture
- Frontend architecture
- Backend architecture
- Scanner architecture
- Worker architecture
- AI architecture
- CBOM schema
- PostgreSQL schema
- Graph schema
- Object storage
- Redis / queues
- API specification
- Authentication APIs
- Scan APIs
- CBOM APIs
- Risk APIs
- Graph APIs
- Recommendation APIs
- Migration APIs
- Report APIs
- AI APIs
- WebSocket/event architecture
- Folder structure
- Repository structure
- Docker architecture
- Kubernetes architecture
- CI/CD
- Environment configuration
- Secrets management
- Database migrations
- Background workers
- Scan orchestration
- Scanner plugin architecture

- Rule engine
- Detection engine
- AST pipeline
- Binary pipeline
- Certificate pipeline
- Container pipeline
- Entity resolution
- CBOM normalization
- Graph construction
- Risk calculation
- Recommendation engine
- AI RAG architecture
- Prompt architecture
- Model gateway
- Testing architecture
- Observability
- Logging
- Metrics
- Deployment profiles
- Development roadmap
- MVP scope
- SIH prototype scope
- Production scope
- Exact implementation order

The key output will be a buildable architecture:





Phase 21 is where the entire 20-phase conceptual design becomes an **actual engineering plan your team can implement.**

Phase 20 complete.